

ООП 1

АБСТРАКТНИ КЛАСОВЕ И ИНТЕРФЕЙСИ.

СЪЩНОСТ

- СЪЩНОСТ НА РОДИТЕЛСКИЯ КЛАС
- ПРИНЦИПА НА АБСТРАКЦИЯТА
 - АБСТРАКТНИ КЛАСОВЕ
 - ИНТЕРФЕЙСИ

АБСТРАКТЕН КЛАС, ХАРАКТЕРИСТИКИ

- ДЕКЛАРАЦИЯ С “ABSTRACT” - `PUBLIC ABSTRACT CLASS .....`
- АБСТРАКТНИ И НЕАБСТРАКТНИ МЕТОДИ
 - АБСТРАКТЕН МЕТОД — ПОНЕ ЕДИН
 - БЕЗ ТЯЛО
 - `ABSTRACT VOID PRINT();`
- НЕ МОЖЕ ДА БЪДЕ ИНСТАНЦИРАН
- МОЖЕ ДА ИМА КОНСТРУКТОРИ И СТАТИЧНИ МЕТОДИ
- МОЖЕ ДА ИМА `FINAL` МЕТОДИ

АБСТРАКТЕН КЛАС ПРИМЕР

```
PUBLIC ABSTRACT CLASS VEHICLE {  
  
    STRING MODEL;  
  
    INT AGE;  
  
  
    PUBLIC VOID PRESENT() {  
        SYSTEM.OUT.PRINTLN("FROM VEHICLE");  
    }  
  
    PUBLIC ABSTRACT VOID RUN();  
}
```


АБСТРАКТЕН КЛАС ПРИМЕР

```
PUBLIC CLASS CAR EXTENDS VEHICLE {  
    STRING TYPE;  
  
    @OVERRIDE  
    PUBLIC VOID RUN() {  
        SYSTEM.OUT.PRINTLN("THE CAR START RUNNING...");  
    }  
}
```

АБСТРАКТЕН КЛАС ПРИМЕР

```
PUBLIC CLASS ABSTRACTIONTEST {  
  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
  
        VEHICLE C1 = NEW CAR();  
        C1.PRESENT();  
        C1.RUN();  
    }  
}
```

From Vehicle
The car start running...

ИНТЕРФЕЙС. СЪЩНОСТ

- АБСТРАКЦИЯ
 - НИСКА ЗАВИСИМОСТ (LOOSE COUPLING)
- КОНТРАКТ ЗА ИМПЛЕМЕНТИРАЩИТЕ КЛАСОВЕ
 - IMPLEMENTS
 - ИМПЛЕМЕНТАЦИЯ НА МНОЖЕСТВО ИНТЕРФЕЙСИ

ИНТЕРФЕЙС. РАЗВИТИЕ

- JAVA 7
 - САМО АБСТРАКТНИ МЕТОДИ И КОНСТАНТИ
- JAVA 8
 - ДЕФОЛТЕН МЕТОД, СТАТИЧНИ МЕТОДИ
- JAVA 9
 - + PRIVATE МЕТОДИ

ИНТЕРФЕЙС. СЪДЪРЖАНИЕ

```
INTERFACE <INTERFACE_NAME>{
```

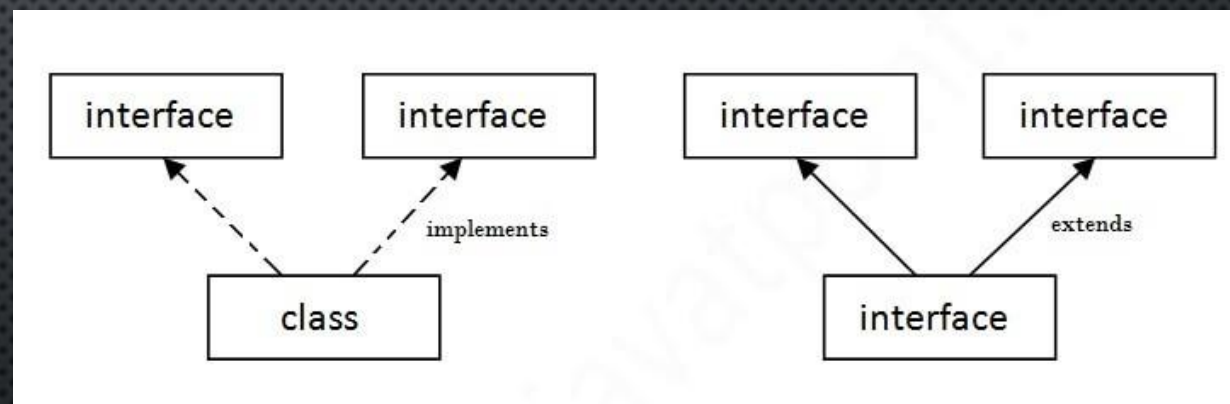
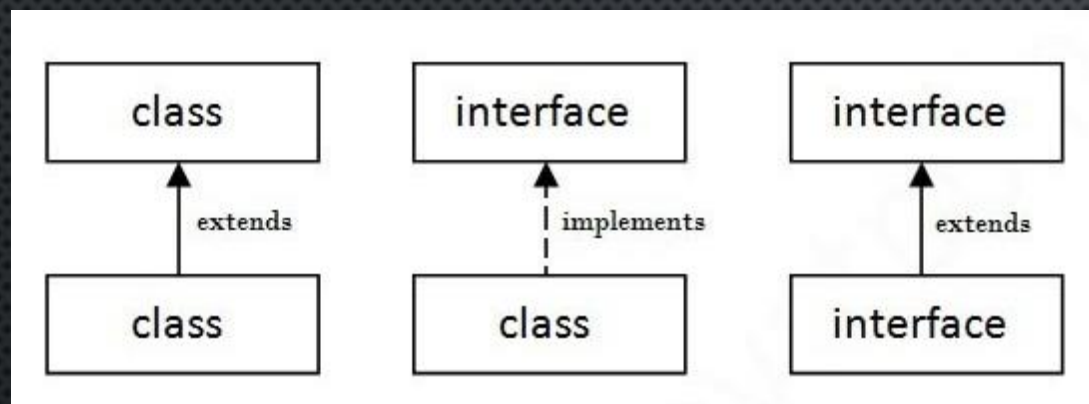
```
    // КОНСТАНТИ
```

```
    // АБСТРАКТНИ И НЕАБСТРАКТНИ МЕТОДИ
```

```
    // ДЕФОЛТЕН МЕТОД
```

```
}
```

ВЗАИМОДЕЙСТВИЯ



ОБЕКТНО КОНВЕРТИРАНЕ (OBJECT CASTING)

- DOWNCASTING
 - Създаване на инстанция на наследник от инстанция на родителя
- `DOG d=new ANIMAL();` - ГРЕШКА ПРИ КОМПИЛАЦИЯ
- `DOG d=(DOG)new ANIMAL();` - ГРЕШКА ПРИ ИЗПЪЛНЕНИЕ (`CLASSCASTEXCEPTION`)

INSTANCEOF OPERATOR

```
IF(A instanceof DOG3){  
    DOG3 D=(DOG3) A  
    .....  
}
```


ТИПОВЕ ИНТЕРФЕЙСИ

- USER-DEFINED
- MARKER
 - SERIALIZABLE, CLONEABLE, REMOTE и др.
 - АНОТАЦИИ КАТО АЛТЕРНАТИВА
- ФУНКЦИОНАЛЕН
 - НАЛИЧИЕ НА ЕДИНСТВЕН АБСТРАКТЕН МЕТОД - SAM(SINGLE ABSTRACT METHOD)
 - LAMBDA
 - `CONSUMER <INTEGER> METHOD = (N) -> { SYSTEM.OUT.PRINTLN (N) ; } ;`

USER-DEFINED

```
PUBLIC INTERFACE ANIMAL {  
    VOID MAKE SOUND();  
}
```

```
PUBLIC CLASS DOG IMPLEMENTS ANIMAL {  
    @OVERRIDE  
    PUBLIC VOID MAKE SOUND() {  
        SYSTEM.OUT.PRINTLN("БАУ!");  
    }  
}
```

```
PUBLIC CLASS CAT IMPLEMENTS ANIMAL {  
    @OVERRIDE  
    PUBLIC VOID MAKE SOUND() {  
        SYSTEM.OUT.PRINTLN("МЯУ!");  
    }  
}
```


МАРКИРАЦІ ІНТЕРФЕЙС

```
PUBLIC INTERFACE DELETABLE {  
    // МАРКИРАЦІ ІНТЕРФЕЙС БЕЗ МЕТОДИ  
}
```

МАРКИРАЩ ИНТЕРФЕЙС

```
PUBLIC CLASS UserAccount IMPLEMENTS Deletable {  
    PRIVATE STRING USERNAME;
```

```
    PUBLIC UserAccount(STRING USERNAME) {  
        THIS.USERNAME = USERNAME;  
    }
```

```
    // ГЕТЪРИ И СЕТЪРИ  
}
```

```
PUBLIC CLASS SYSTEMCONFIG {  
    PRIVATE STRING CONFIGNAME;
```

```
    PUBLIC SYSTEMCONFIG(STRING CONFIGNAME) {  
        THIS.CONFIGNAME = CONFIGNAME;  
    }
```

```
    // ГЕТЪРИ И СЕТЪРИ  
}
```


МАРКИРАЩ ИНТЕРФЕЙС

```
PUBLIC CLASS DeletionService {  
    PUBLIC VOID DeleteObject(Object obj) {  
        IF (obj instanceof Deletable) {  
            // ИЗТРИВАНЕ НА ОБЕКТА  
            SYSTEM.OUT.PRINTLN("Обектът " + obj.getClass().getSimpleName() + " е изтрил.");  
        } ELSE {  
            SYSTEM.OUT.PRINTLN("Обектът " + obj.getClass().getSimpleName() + " не може  
да бъде изтрил.");  
        }  
    }  
}
```

```
PUBLIC STATIC VOID MAIN(String[] args) {  
    DeletionService service = new DeletionService();  
  
    UserAccount user = new UserAccount("IVAN_PETROV");  
    SystemConfig config = new SystemConfig("SYSTEM SETTINGS");  
  
    service.DeleteObject(user); // Обектът ще бъде изтрил  
    service.DeleteObject(config); // Обектът няма да бъде изтрил  
}
```

Обектът UserAccount е изтрил.
Обектът SystemConfig не може да бъде изтрил.

МАРКИРАЩ ИНТЕРФЕЙС

- **МАРКИРАЩИЯТ ИНТЕРФЕЙС Deletable:** Използва се за обозначаване на обекти, които могат да бъдат изтрити.
- **Класът UserAccount:** Имплементира Deletable, което означава, че обектите му могат да бъдат изтрити.
- **Класът SystemConfig:** Не имплементира Deletable, тъй като обектите му не трябва да бъдат изтрити.
- **Класът DeletionService:** Съдържа метод DeleteObject, който проверява дали обектът имплементира Deletable и съответно решава дали да го изтрие.

ФУНКЦИОНАЛЕН ИНТЕРФЕЙС SAM(SINGLE ABSTRACT METHOD)

```
@FunctionalInterface  
  
public interface Calculator {  
    int calculate(int a, int b);  
}
```

- **АНОТАЦИЯТА @FunctionalInterface:** Тази анотация указва на компилатора, че интерфейсът е предназначен да бъде функционален. Ако добавите повече от един абстрактен метод, компилаторът ще даде грешка.
- **АБСТРАКТЕН МЕТОД CALCULATE:** Единственият абстрактен метод в интерфейса, който приема два цели числа и връща резултат.

ФУНКЦИОНАЛЕН ИНТЕРФЕЙС SAM(SINGLE ABSTRACT METHOD)

```
PUBLIC CLASS FUNCTIONALINTERFACEDEMO {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        // ИМПЛЕМЕНТАЦИЯ ЗА СЪБИРАНЕ  
        CALCULATOR ADDITION = (A, B) -> A + B;  
        SYSTEM.OUT.PRINTLN("СЪБИРАНЕ: " + ADDITION.CALCULATE(10, 5));  
  
        // ИМПЛЕМЕНТАЦИЯ ЗА ИЗВАЖДАНЕ  
        CALCULATOR SUBTRACTION = (A, B) -> A - B;  
        SYSTEM.OUT.PRINTLN("ИЗВАЖДАНЕ: " + SUBTRACTION.CALCULATE(10, 5));  
    }  
}  
  
// ИМПЛЕМЕНТАЦИЯ ЗА УМНОЖЕНИЕ  
CALCULATOR MULTIPLICATION = (A, B) -> A * B;  
SYSTEM.OUT.PRINTLN("УМНОЖЕНИЕ: " + MULTIPLICATION.CALCULATE(10, 5));  
  
// ИМПЛЕМЕНТАЦИЯ ЗА ДЕЛЕНИЕ  
CALCULATOR DIVISION = (A, B) -> A / B;  
SYSTEM.OUT.PRINTLN("ДЕЛЕНИЕ: " + DIVISION.CALCULATE(10, 5));
```

Събиране: 15
Изваждане: 5
Умножение: 50
Деление: 2

ФУНКЦИОНАЛЕН ИНТЕРФЕЙС SAM(SINGLE ABSTRACT METHOD)

- **ЛАМБДА ИЗРАЗИ:** ИЗПОЛЗВАМЕ ЛАМБДА ИЗРАЗИ ЗА ИМПЛЕМЕНТИРАНЕ НА МЕТОДА CALCULATE ЗА РАЗЛИЧНИ МАТЕМАТИЧЕСКИ ОПЕРАЦИИ. $(A, B) \rightarrow A + B$ Е ИМПЛЕМЕНТАЦИЯ ЗА СЪБИРАНЕ.
- $(A, B) \rightarrow A - B$ Е ИМПЛЕМЕНТАЦИЯ ЗА ИЗВАЖДАНЕ.
- И Т.Н.
- **ИЗПОЛЗВАНЕ НА ФУНКЦИОНАЛНИЯ ИНТЕРФЕЙС:** СЪЗДАВАМЕ РАЗЛИЧНИ ИНСТАНЦИИ НА CALCULATOR, ВСЯКА С РАЗЛИЧНА ИМПЛЕМЕНТАЦИЯ НА МЕТОДА CALCULATE.

ПОЛИМОРФИЗЪМ

- ПОЛИМОРФИЗЪМЪТ Е СПОСОБНОСТТА НА ОБЕКТИ ОТ РАЗЛИЧНИ КЛАСОВЕ, КОИТО СА НАСЛЕДНИЦИ НА ЕДИН И СЪЩ БАЗОВ КЛАС, ДА БЪДАТ ТРЕТИРАНИ КАТО ОБЕКТИ ОТ ТОЗИ БАЗОВ КЛАС, НО ДА СЕ ДЪРЖАТ ПО СПЕЦИФИЧЕН ЗА ТЯХ НАЧИН. ТОВА ОЗНАЧАВА, ЧЕ ЕДИН И СЪЩ КОД МОЖЕ ДА РАБОТИ С РАЗЛИЧНИ ОБЕКТИ И ДА ИЗПЪЛНЯВА ПОДХОДЯЩИТЕ ЗА ТЯХ ДЕЙСТВИЯ.

ВИДОВЕ ПОЛИМОРФИЗЪМ

- СТАТИЧЕН – ПО ВРЕМЕ НА КОМПИЛАЦИЯ
 - `STATIC`
 - `DOG d1=new DOG();`
- ДИНАМИЧЕН – ПО ВРЕМЕ НА ИЗПЪЛНЕНИЕ
 - `ANIMAL A= NEW DOG();`

КАКВИ ПРОБЛЕМИ РЕШАВА ПОЛИМОРФИЗМЪТ

- **ГЪВКАВОСТ И РАЗШИРЯЕМОСТ НА КОДА:**
 - **ОПИСАНИЕ:** ПОЛИМОРФИЗМЪТ ПОЗВОЛЯВА ДА СЕ ПИШЕ КОД, КОЙТО РАБОТИ С БАЗОВИ ТИПОВЕ ИЛИ ИНТЕРФЕЙСИ, БЕЗ ДА СЕ ИНТЕРЕСУВА ОТ КОНКРЕТНАТА РЕАЛИЗАЦИЯ. ТОВА ПРАВИ КОДА ПО-ГЪВКАВ И ЛЕСЕН ЗА РАЗШИРЯВАНЕ.
 - **ПРОБЛЕМ, КОЙТО РЕШАВА:** НАМАЛЯВА НУЖДАТА ОТ ПИСАНЕ НА СПЕЦИФИЧЕН КОД ЗА ВСЕКИ НОВ ТИП ОБЕКТ, КОЕТО УЛЕСНЯВА ДОБАВЯНЕТО НА НОВИ КЛАСОВЕ БЕЗ ПРОМЯНА НА СЪЩЕСТВУВАЩИЯ КОД.

КАКВИ ПРОБЛЕМИ РЕШАВА ПОЛИМОРФИЗМЪТ

- **НАМАЛЯВА СЛОЖНОСТТА И ПОДОБРЯВА ЧЕТИМОСТТА:**
 - **ОПИСАНИЕ:** ВМЕСТО ДА ИЗПОЛЗВАМЕ СЛОЖНИ УСЛОВНИ КОНСТРУКЦИИ КАТО IF-ELSE ИЛИ SWITCH ЗА ПРОВЕРКА НА ТИПА НА ОБЕКТА, МОЖЕМ ДА ИЗПОЛЗВАМЕ ПОЛИМОРФИЗЪМ ЗА ИЗВИКВАНЕ НА ПОДХОДЯЩИЯ МЕТОД.
 - **ПРОБЛЕМ, КОЙТО РЕШАВА:** ПРЕДОТВРАТЯВА РАЗРАСТВАНЕТО НА УСЛОВНИ ИЗРАЗИ И УЛЕСНЯВА ПОДДРЪЖКАТА НА КОДА.

КАКВИ ПРОБЛЕМИ РЕШАВА ПОЛИМОРФИЗМЪТ

- **УЛЕСНЯВА ПОДДРЪЖКАТА И ТЕСТВАНЕТО:**
 - **ОПИСАНИЕ:** КОГАТО КОДЪТ Е НАПИСАН ПОЛИМОРФИЧНО, Е ПО-ЛЕСНО ДА СЕ ТЕСТВАТ ОТДЕЛНИ КОМПОНЕНТИ И ДА СЕ ЗАМЕНЯТ РЕАЛНИ ОБЕКТИ С ТЕСТОВИ ДВОЙКИ (MOCK OBJECTS).
 - **ПРОБЛЕМ, КОЙТО РЕШАВА:** УЛЕСНЯВА ПРОЦЕСА НА ТЕСТВАНЕ И ОТКРИВАНЕ НА ГРЕШКИ.
- **ПОДДЪРЖА ПРИНЦИПИТЕ НА ООП:**
 - **ОПИСАНИЕ:** ПОЛИМОРФИЗМЪТ НАСЪРЧАВА ИЗПОЛЗВАНЕТО НА АБСТРАКЦИИ И ИНТЕРФЕЙСИ, КОЕТО ВОДИ ДО ПО-ДОБРА АРХИТЕКТУРА НА СОФТУЕРА.
 - **ПРОБЛЕМ, КОЙТО РЕШАВА:** ПОЗВОЛЯВА ИЗГРАЖДАНЕТО НА ПО-ГЪВКАВИ И МАЩАБИРУЕМИ СИСТЕМИ.

МЕТОДНО ПРЕДЕФИНИРАНЕ (OVERRIDING)

```
PUBLIC CLASS ANIMAL {  
    PUBLIC VOID MAKE SOUND() {  
        SYSTEM.OUT.PRINTLN("ЖИВОТНОТО ИЗДАВА ЗВУК.");  
    }  
}
```

```
PUBLIC CLASS DOG EXTENDS ANIMAL {  
    @OVERRIDE  
    PUBLIC VOID MAKE SOUND() {  
        SYSTEM.OUT.PRINTLN("КУЧЕТО КАЗВА: БАУ!");  
    }  
}
```

```
PUBLIC CLASS CAT EXTENDS ANIMAL {  
    @OVERRIDE  
    PUBLIC VOID MAKE SOUND() {  
        SYSTEM.OUT.PRINTLN("КОТКАТА КАЗВА: МЯУ!");  
    }  
}
```

```
}  
}
```

```
PUBLIC CLASS MAIN {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        ANIMAL MYANIMAL = NEW ANIMAL(); // СЪЗДАВАМЕ ОБЕКТ ОТ ТИП ANIMAL  
        ANIMAL MYDOG = NEW DOG();      // ПОЛИМОРФИЗЪМ: ОБЕКТ ОТ ТИП DOG СЕ  
        ТРЕТИРА КАТО ANIMAL  
        ANIMAL MYCAT = NEW CAT();      // ПОЛИМОРФИЗЪМ: ОБЕКТ ОТ ТИП CAT СЕ ТРЕТИРА  
        КАТО ANIMAL  
  
        MYANIMAL.MAKE SOUND(); // ИЗВИКВА ANIMAL.MAKE SOUND()  
        MYDOG.MAKE SOUND();    // ИЗВИКВА DOG.MAKE SOUND()  
        MYCAT.MAKE SOUND();    // ИЗВИКВА CAT.MAKE SOUND()  
    }  
}
```

Животното издава звук.
Кучето казва: Бау!
Котката казва: Мяу!

КАКВИ ПРОБЛЕМИ РЕШАВА ПОЛИМОРФИЗМЪТ

- Въпреки че myDog и myCat са от тип Animal, при извикване на makeSound() се извикват съответните методи на Dog и Cat поради динамичния полиморфизъм.
- Това позволява да работим с общия тип Animal, без да се интересуваме от конкретния тип животно.

МЕТОДНО ПРЕТОВАРВАНЕ (OVERLOADING)

```
PUBLIC CLASS MATHOPERATIONS {  
    // МЕТОД ЗА СЪБИРАНЕ НА ДВЕ ЧИСЛА  
    PUBLIC INT ADD(INT A, INT B) {  
        RETURN A + B;  
    }  
  
    // МЕТОД ЗА СЪБИРАНЕ НА ТРИ ЧИСЛА  
    PUBLIC INT ADD(INT A, INT B, INT C) {  
        RETURN A + B + C;  
    }  
  
    // МЕТОД ЗА СЪБИРАНЕ НА ДВЕ ДРОБНИ ЧИСЛА  
    PUBLIC DOUBLE ADD(DOUBLE A, DOUBLE B) {  
        RETURN A + B;  
    }  
}
```

```
PUBLIC CLASS MAIN {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        MATHOPERATIONS MATH = NEW MATHOPERATIONS();  
  
        INT SUM1 = MATH.ADD(5, 10);  
        INT SUM2 = MATH.ADD(5, 10, 15);  
        DOUBLE SUM3 = MATH.ADD(5.5, 10.5);  
  
        SYSTEM.OUT.PRINTLN("СУМА1: " + SUM1);  
        SYSTEM.OUT.PRINTLN("СУМА2: " + SUM2);  
        SYSTEM.OUT.PRINTLN("СУМА3: " + SUM3);  
    }  
}
```

Сума1: 15
Сума2: 30
Сума3: 16.0

МЕТОДНО ПРЕТОВАРВАНЕ (OVERLOADING)

- МЕТОДЪТ ADD Е ПРЕТОВАРЕН С РАЗЛИЧНИ ПАРАМЕТРИ.
- КОМПИЛАТОРЪТ РЕШАВА КОЙ МЕТОД ДА ИЗВИКА ВЪЗ ОСНОВА НА БРОЯ И ТИПА НА АРГУМЕНТИТЕ.

ИЗПОЛЗВАНЕ НА ИНТЕРФЕЙСИ И ПОЛИМОРФИЗЪМ

```
PUBLIC INTERFACE ANIMAL {  
    VOID MAKESOUND();  
}
```

```
PUBLIC CLASS DOG IMPLEMENTS ANIMAL {  
    @OVERRIDE  
    PUBLIC VOID MAKESOUND() {  
        SYSTEM.OUT.PRINTLN("БАУ!");  
    }  
}
```

```
PUBLIC CLASS CAT IMPLEMENTS ANIMAL {  
    @OVERRIDE  
    PUBLIC VOID MAKESOUND() {  
        SYSTEM.OUT.PRINTLN("Мяу!");  
    }  
}
```

```
PUBLIC CLASS COW IMPLEMENTS ANIMAL {  
    @OVERRIDE  
    PUBLIC VOID MAKESOUND() {  
        SYSTEM.OUT.PRINTLN("Му!");  
    }  
}
```

```
PUBLIC CLASS FARM {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        // Създаваме масив от животни  
        ANIMAL[] ANIMALS = { NEW DOG(), NEW CAT(), NEW COW() };  
  
        // Извикваме метода MAKESOUND() за всяко животно  
        FOR (ANIMAL ANIMAL : ANIMALS) {  
            ANIMAL.MAKESOUND(); // Динамичен байндинг  
        }  
    }  
}
```

ПРАКТИЧЕСКИ ПОЛЗИ ОТ ПОЛИМОРФИЗМА

- БЕЗ ПОЛИМОРФИЗЪМ:

```
for (Object obj : objects) {  
    if (obj instanceof Dog) {  
        ((Dog) obj).makeSound();  
    } else if (obj instanceof Cat) {  
        ((Cat) obj).makeSound();  
    }  
    // И така нататък за всеки тип  
}
```

- С ПОЛИМОРФИЗЪМ:

```
for (Animal animal : animals) {  
    animal.makeSound();  
}
```


ПРАКТИЧЕСКИ ПОЛЗИ ОТ ПОЛИМОРФИЗМА

- **УЛЕСНЕНО ДОБАВЯНЕ НА НОВИ КЛАСОВЕ**
 - МОЖЕТЕ ДА ДОБАВИТЕ НОВ КЛАС SHEEP, КОЙТО ИМПЛЕМЕНТИРА ANIMAL, БЕЗ ДА ПРОМЕНЯТЕ КОДА В FARM.
- **ИЗПОЛЗВАНЕ НА ОБЩИ АЛГОРИТМИ**
 - МОЖЕТЕ ДА РАЗРАБОТИТЕ АЛГОРИТМИ, КОИТО РАБОТЯТ С АБСТРАКТНИ ТИПОВЕ, И ДА ГИ ПРИЛАГАТЕ КЪМ РАЗЛИЧНИ КОНКРЕТНИ РЕАЛИЗАЦИИ.